

```

module hazard_unit(
    input wire [4:0] is_Rs1,    // Source register 1 in ID stage
    input wire [4:0] is_Rs2,    // Source register 2 in ID stage
    input wire [4:0] a1, a2,    // Register addresses used for forwarding checks
    input wire ex_regsrsc,      // EX stage register source valid signal
    input wire m_regsrsc,      // MEM stage register source valid signal
    input wire [4:0] id_rd,     // Destination register in ID stage
    input wire [4:0] ex_rd,     // Destination register in EX stage
    input wire [4:0] mem_rd,    // Destination register in MEM stage
    input wire load,           // Load instruction signal
    input wire amo_en_ex,      // Atomic operation in EX stage
    input wire amo_en_mem,     // Atomic operation in MEM stage
    input wire amo_en_id,      // Atomic operation in ID stage
    output wire load_use_hazard,
    output wire [1:0] mux1_sel, // Forwarding selection for source 1
    output wire [1:0] mux2_sel  // Forwarding selection for source 2
);

// Load-Use Hazard Detection: If a load instruction or AMO is in ID stage, stall
assign load_use_hazard = ((is_Rs1 == id_rd) || (is_Rs2 == id_rd)) && (load ||
amo_en_id);

// Forwarding Logic for Source 1 (Rs1)
assign mux1_sel = (ex_regsrsc && (ex_rd != 0) && (a1 == ex_rd)) ? 2'b01 : // Forward
from EX stage
                (m_regsrsc && (mem_rd != 0) && (a1 == mem_rd)) ? 2'b10 : // Forward
from MEM stage
                (amo_en_ex && (ex_rd != 0) && (a1 == ex_rd)) ? 2'b01 : // Forward
from AMO in EX
                (amo_en_mem && (mem_rd != 0) && (a1 == mem_rd)) ? 2'b10 : //
Forward from AMO in MEM
                2'b00; // No forwarding

// Forwarding Logic for Source 2 (Rs2)
assign mux2_sel = (ex_regsrsc && (ex_rd != 0) && (a2 == ex_rd)) ? 2'b01 : // Forward
from EX stage
                (m_regsrsc && (mem_rd != 0) && (a2 == mem_rd)) ? 2'b10 : // Forward
from MEM stage
                (amo_en_ex && (ex_rd != 0) && (a2 == ex_rd)) ? 2'b01 : // Forward
from AMO in EX
                (amo_en_mem && (mem_rd != 0) && (a2 == mem_rd)) ? 2'b10 : //
Forward from AMO in MEM
                2'b00; // No forwarding

endmodule

```